

Технический сбор Telegram Mini App

План разработки, архитектура, UX/UI, модули, приемка и эксплуатация.

Статус документа

Рабочая версия. Основана на бизнес-плане Adnet и процессных схемах проекта.

Telegram-first marketplace для стандартизированных рекламных размещений и UGC-заказов.

Подготовлено: июль 2026

1. Назначение и границы продукта

Adnet - Telegram-first marketplace, где рекламодатель создает стандартизированную кампанию, исполнители подают заявки, бюджет резервируется у платежного провайдера, результат подтверждается доказательствами, а затем проходит приемку, выплату или спор.

Главная продуктовая цель MVP

Позволить вручную или гибридно провести безопасную сделку от кампании до выплаты и финального отчета. Автоматизация платформ на первом этапе не является обещанием пользователю.

Что входит в MVP

- Telegram-аутентификация и роли: рекламодатель, исполнитель, администратор.
- Кампании двух типов: размещение на площадке и производство UGC-файла.
- Пять коммерчески доступных платформ: Telegram, VK, YouTube, TikTok, Instagram. Проверка на старте может быть ручной или гибридной.
- Заявки, выбор исполнителя, карточка сделки со строгими статусами.
- Внесение результата, Evidence, приемка, доработка, спор, статус платежа и аудит действий.
- Мини-приложение Telegram для основных ролей и отдельная web-admin панель.

Что не входит в MVP

- Нативные мобильные приложения iOS/Android.
- Внутренний P2P-баланс и самостоятельное хранение денег Adnet.
- Гарантия охвата, продаж, лидов или рекламного результата.
- Полная автоматическая интеграция со всеми платформами.
- Сложные кабинетные функции агентств, подписки и performance-атрибуция.

2. Принципы реализации

Принцип	Практическое решение
Вертикальный срез	Первый релиз проводит одну сделку целиком: кампания -> заявка -> сделка -> результат -> ручная приемка.
Статус как источник истины	Нельзя менять этап сделки только интерфейсом. Переходы валидируются сервером и записываются в AuditLog.
Ручной fallback	Если API площадки или платежа недоступен, модератор видит задачу и принимает решение по чек-листу.
Доказательства отдельно от решения	Evidence хранит ссылку, файл, скриншот и метаданные; Review хранит решение и комментарий.
Не хранить деньги	Adnet хранит идентификаторы и статусы внешнего платежа; удержание и payout выполняет лицензированный провайдер.
Telegram-first	Мини-приложение решает основную работу, бот присылает уведомления и запускает приложение из контекста.

3. Пользователи и ключевые сценарии

Роль	Задача	Ключевой сценарий
Рекламодатель	Получить контролируемое размещение или UGC	Создает кампанию, выбирает исполнителя, резервирует бюджет, принимает результат, смотрит отчет.
Исполнитель	Находить понятные оплачиваемые задания	Заполняет профиль, привязывает площадку, подает заявку, выполняет ТЗ, передает результат, получает выплату.
Модератор	Снижать риски и проводить исключения	Проверяет кампанию, аккаунты, Evidence, спор и соответствие правилам.
Оператор	Поддерживать запуск рынка	Сопровождает первые кампании, уточняет ТЗ, помогает с подбором и эскалирует проблемы.

Сквозной сценарий

Campaign -> Application -> Deal -> Submission -> Review / Dispute -> Payment. Этот контур является основой продукта: все модули должны усиливать его, а не существовать отдельно.

Два типа сделки

Тип	Результат	Критерий приемки
Placement	Публичная публикация на площадке исполнителя	Ссылка доступна, площадка и аккаунт совпадают, соблюдены ТЗ/маркировка/хэштег, контент не удален в контрольный период.
UGC production	Файл для использования рекламодателем	Файл доступен, формат и содержание соответствуют ТЗ, права и версии зафиксированы.

4. Техническая архитектура

Рекомендуемая архитектура: один репозиторий с несколькими приложениями и общими типами. На MVP допустим модульный монолит; выделять микросервисы до реальной нагрузки не нужно.

Слой	Решение	Ответственность
Telegram bot	TypeScript + grammY или Telegraf	Аутентификация, deep links, уведомления, открытие Mini App.
Mini App	Next.js + React + TypeScript	Интерфейс рекламодателя и исполнителя; Telegram WebApp SDK.
Admin web	Next.js + TypeScript	Модерация, Evidence, споры, статусы платежей, аудит.
Backend API	NestJS + TypeScript	RBAC, бизнес-правила, state machine, REST API, webhooks.
Database	PostgreSQL	Транзакционные сущности, индексы, журнал аудита.
Queue	Redis + BullMQ	Уведомления, контрольные сроки, повторные webhook-задачи, сбор метрик.
File storage	S3-compatible storage	UGC-файлы, скриншоты, файлы Evidence, подписанные URL.
Observability	Sentry + structured logs	Ошибки, трассировка ключевых действий, алерты.
Payments	Внешний marketplace-провайдер	Checkout, reserve/hold, payout, refund, webhook-статусы.

Граница надежности

Клиентское приложение никогда не определяет окончательный статус сделки или платежа. Сервер принимает решение на основе разрешенного перехода, роли и подтвержденного события.

5. Модули MVP и Definition of Done

Модуль	Минимум в MVP	Готово, когда
Auth и RBAC	Telegram initData, сессия, роли, защита API.	Нельзя подменить пользователя; каждая роль видит только свои данные и действия.
Профили	Компания, creator profile, площадки, услуги, реквизиты для payout.	Профиль можно заполнить/изменить, площадку отправить на проверку.
Campaigns	Создание, черновик, модерация, публикация, список.	Кампания имеет обязательное ТЗ, бюджет, дедлайн, тип и статусы.
Applications	Подача, отзыв, отклонение, принятие.	Нельзя принять две заявки, если кампания рассчитана на одного исполнителя.
Deals	Карточка, участники, чек-лист, сроки, state machine.	Все переходы валидны, идемпотентны и попадают в аудит.
Submission/Evidence	Ссылки, файлы, скриншоты, комментарии, версии.	Результат не теряется; рекламодатель/модератор видят все доказательства.
Review/Dispute	Принять, запросить доработку, открыть спор, решение модератора.	Решение имеет автора, причину, timestamp и влияет на следующий допустимый статус.
Payments	Создать checkout, получить webhook, показать статусы reserve/payout/refund.	Дубликаты webhook не создают дублирующих выплат; все расхождения видны админу.
Notifications	Bot и in-app события по важным переходам.	Пользователь получает понятное действие, ссылку на нужную сущность и не получает дубли.
Admin/Audit	Очереди проверок, фильтры, ручные решения, AuditLog.	Админ может восстановить историю решения по любой сделке.

6. Статусы и правила сделок

Состояния должны быть реализованы как конечный автомат. В базе допускается хранить статус строкой, но переходы следует централизовать в одном сервисе с таблицей разрешений.

Статус	Кто переводит	Следующие состояния	Смысл
draft	system / advertiser	awaiting_funding, cancelled	Сделка подготовлена после принятия заявки.
awaiting_funding	advertiser / payment webhook	funds_reserved, cancelled	Ожидается успешный reserve у провайдера.
funds_reserved	system	in_progress	Бюджет подтвержденно зарезервирован.
in_progress	performer	submitted, cancelled	Исполнитель выполняет ТЗ до дедлайна.
submitted	advertiser / moderator	revision_requested, under_review, accepted, disputed	Передан результат и Evidence.
revision_requested	performer	submitted, disputed	Требуется доработка с комментарием.
under_review	advertiser / moderator	accepted, disputed	Идет контрольный период или проверка.
accepted	system / payment webhook	payout_pending	Результат принят, запуск выплаты разрешен.
payout_pending	payment webhook	paid, disputed	Провайдер обрабатывает payout.
paid	system	closed	Исполнитель получил выплату.

Статус	Кто переводит	Следующие состояния	Смысл
disputed	moderator	accepted, refund_pending, closed	Открыт спор, обычные переходы заблокированы.
refund_pending	payment webhook	refunded, closed	Провайдер обрабатывает полный или частичный возврат.
closed	system	-	Сделка завершена, данные доступны только для чтения.

Обязательные инварианты

- У сделки ровно один рекламодатель; исполнитель появляется после принятия заявки.
- Выплата возможна только после accepted и подтвержденного резервирования.
- Спор фиксирует снимок данных на момент открытия: T3, Evidence, сообщения, статусы.
- Отмена после funds_reserved всегда требует сценария возврата или решения модератора.
- Каждый переход пишет DealEvent и AuditLog с actor, source и correlation id.

7. Модель данных

Сущность	Существенные поля	Связи
User	id, telegram_id, role, status, locale	1:N profiles, audit entries
AdvertiserCompany	legal_name, contacts, verification_status	1:N campaigns
CreatorProfile	bio, categories, rate_hint, payout_profile	1:N social accounts, applications
SocialAccount	platform, handle, url, verification_status, metrics_snapshot	N:1 creator
Campaign	type, brief, budget, deadline, state, moderation_status	N:1 company; 1:N applications/deals
Application	price, message, proposed terms, status	N:1 campaign, N:1 creator
Deal	status, amount, commission, deadlines, accepted_application_id	N:1 campaign, N:1 creator
Submission	version, submitted_at, comment, status	N:1 deal; 1:N evidence
Evidence	kind, url/storage_key, metadata, captured_at	N:1 submission
Review	decision, checklist, comment, actor_id	N:1 deal/submission
Payment	provider, external_id, status, amount, kind, raw_event_ref	N:1 deal
Dispute	reason, status, resolution, resolver_id	N:1 deal
AuditLog	entity_type, entity_id, action, actor, payload, timestamp	append-only

Требования к данным

- Деньги хранятся в целых копейках/центах и валюте; никогда не float.
- Время хранится в UTC, клиент получает локальное представление.
- Публичные URL файлов не постоянны: выдавать short-lived signed URLs.
- Удаление пользовательских данных - soft delete; Evidence и AuditLog имеют отдельные сроки хранения.
- Webhook-пакеты хранить ограниченно и маскировать персональные/платежные данные.

8. API и интеграции

Для Mini App использовать REST JSON API с OpenAPI-спецификацией. Внутренние события в очереди должны иметь версию схемы. Внешние webhooks обрабатываются отдельными endpoint-ами и подтверждаются только после идемпотентной записи.

Группа	Примеры endpoint-ов	Правило
Session	POST /auth/telegram, GET /me	Проверять Telegram initData подпись на сервере.
Campaigns	GET/POST /campaigns, GET/PATCH /campaigns/:id	Редактирование после публикации ограничено версией или возвратом на модерацию.
Applications	POST /campaigns/:id/applications, POST /applications/:id/accept	Принятие заявки атомарно создает/обновляет deal.
Deals	GET /deals/:id, POST /deals/:id/submit, POST /deals/:id/review	Каждая команда содержит idempotency key.
Files	POST /uploads/presign, POST /evidence	Клиент загружает прямо в storage по подписанной ссылке.
Payments	POST /deals/:id/checkout, POST /webhooks/payment	Сверять подпись, external id и допустимый переход.
Admin	GET /admin/review-queue, POST /admin/disputes/:id/resolve	RBAC, причина ручного решения обязательна.

Telegram Mini App

- При первом открытии Mini App отправляет initData на backend; backend валидирует подпись Telegram и создает сессию.
- Использовать MainButton только для главного действия экрана; перед критическим подтверждением показывать Telegram confirmation popup.
- Deep links бота должны открывать конкретную кампанию, сделку или задачу проверки.
- Уведомления: новая заявка, выбрали исполнителя, нужен reserve, результат подан, нужна доработка, решение спора, выплата.
- Нельзя полагаться только на push бота: важные события видны в списке активностей в Mini App.

9. UX/UI требования

Продуктовое ощущение
Adnet должен ощущаться как спокойный рабочий инструмент для сделки, а не как лента развлечений. Ключевые данные: статус, сумма, дедлайн, следующее действие, история.

Информационная архитектура

Роль	Главные экраны	Главный СТА
Рекламодатель	Мои кампании, создать кампанию, заявки, сделка, отчет, компания	Создать кампанию / принять результат
Исполнитель	Лента задач, задача, мои сделки, передать результат, профиль	Откликнуться / передать результат
Администратор	Очередь модерации, кампании, Evidence, споры, платежи, аудит	Принять решение

Правила интерфейса Mini App

- Проектировать mobile-first в ширине Telegram WebView; базовая вертикальная зона действия не должна перекрываться safe area.
- В каждой карточке сделки всегда видны статус, сумма, дедлайн, контрагент и одно следующее действие.
- Не скрывать условия ТЗ за несколькими экранами: summary, вложения и критерии приемки доступны в карточке сделки.
- Сложное создание кампании делать пошагово: тип услуги -> площадка/требования -> ТЗ -> бюджет/срок -> просмотр -> отправка на модерацию.
- Входящие заявки сравнивать в одной структуре: цена, срок, площадка, метрики, рейтинг, комментарий.
- Использовать Telegram theme params, нативные цвета и haptic feedback только на значимых подтверждениях.
- Для ошибок давать причину и конкретное восстановительное действие; не показывать технические коды пользователю.
- Все destructive actions требуют подтверждения и объясняют финансовое последствие.

Состояния экрана

Состояние	Что должно быть
Loading	Скелетон фиксированного размера, без прыжков интерфейса.
Empty	Контекстная причина и один понятный СТА: создать кампанию, заполнить профиль, изменить фильтр.
Error	Краткое объяснение, retry и безопасный путь назад.
Restricted	Почему действие недоступно: модерация, роль, платеж, дедлайн или статус сделки.
Offline/timeout	Сохранить локальный черновик, показать время последней синхронизации, не имитировать успех.

10. Проверка результата, платежа, спор

Evidence checklist

- Placement: ссылка, площадка, handle/аккаунт, дата, промокод/хэштег/маркировка при необходимости, скриншот, публичные метрики.
- UGC: исходный или финальный файл, формат, версия, комментарий исполнителя, права на использование, при необходимости ссылка на облачное хранилище.
- Контрольный период: расписание повторной проверки не должен зависеть от открытого приложения пользователя.
- Проверка имеет результат pass/fail/manual_review и содержит отдельные пункты чек-листа.

Платежный контур

Событие	Действие Adnet	Результат
Выбор исполнителя	Создать deal и checkout request	Рекламодатель видит сумму и комиссию.
Провайдер подтвердил reserve	Сверить подпись и external id, зафиксировать payment	Deal -> funds_reserved -> in_progress.
Принят результат	Создать payout request	Deal -> payout_pending.
Провайдер подтвердил payout	Зафиксировать payment event	Deal -> paid -> closed.
Возврат/спор	Создать refund/partial refund после решения	Сумма и основание видны обоим сторонам.

Критично для разработки

Webhook нельзя обрабатывать "в лоб". Требуется проверка подписи, дедупликация по event id, транзакция базы, повторяемость операции и алерт на неподходящий статус.

11. Безопасность, качество и эксплуатация

Направление	Минимальное требование MVP
Доступ	Проверка Telegram initData, короткая серверная сессия, RBAC на каждом endpoint.
Платежи	Не логировать полные платежные данные; проверять webhook signatures; ручная сверка исключений.
Файлы	Антивирус/проверка MIME при возможности, приватный bucket, signed URLs, лимиты размера и типа.
Аудит	Append-only AuditLog для финансовых решений, модерации и статусов сделки.
Наблюдаемость	Sentry, structured logs, request/correlation id, алерты на неуспешные webhooks и фоновые задачи.
Резервное копирование	Автоматические backup PostgreSQL и периодическая проверка восстановления.
Производительность	Индексы на статус/дедлайн/внешние ids; пагинация всех списков; фоновые сборы метрик через queue.

Тестирование

- Unit: state machine, расчет комиссии, права доступа, idempotency, маппинг webhook.
- Integration: создание кампании, принятие заявки, reserve, submission, review, payout/refund.
- E2E: два аккаунта и admin проходят сценарии placement и UGC.
- Contract: схемы API и webhooks платежного провайдера.
- UX regression: ключевые экраны Mini App в узкой ширине Telegram WebView.
- Ручной чек-лист перед релизом: новая кампания, новая заявка, приемка, спор, возврат, уведомления, админ-аудит.

12. Порядок разработки и приемка

Каждый модуль декомпозируется не только на backend/frontend. Для любой функции создается один короткий пакет: сценарии -> данные -> API -> UI -> уведомления -> админ-действия -> тесты -> критерии готовности.

Этап	Результат	Приемочный сценарий
R0. Foundation	Мониторинг, окружения, CI, auth, роли, база, аудит.	Пользователь входит через Telegram и видит корректный интерфейс своей роли.
R1. Deal slice	Campaign, application, deal, submission, ручная приемка без реального reserve.	Команда проводит тестовую сделку end-to-end.
R2. Safe transaction	Payment statuses, evidence checklist, revision/dispute, admin queues.	Тестовая сделка проходит payout или controlled refund.
R3. Private beta	Уведомления, отчеты, надежность, продуктовая аналитика.	10-30 реальных сделок проведены с измеримыми ручными затратами.
R4. Automation	Адаптеры площадок и фоновые проверки там, где это надежно.	Автоматизация уменьшает ручное время, не ухудшая качество решения.

Шаблон постановки задачи разработчику

- Проблема и пользователь, для которого это делается.
- Сценарий: кто начинает, что делает, какой результат получает.
- Правила и крайние случаи, включая права ролей и дедлайны.
- Изменения данных, API, экранов, уведомлений и админ-панели.
- Критерии приемки в формате проверяемых примеров.
- Аналитические события и тесты, которые должны появиться вместе с функцией.

Первое решение после старта

Не начинать с ленты, рейтингов или автоматизации площадок. Первый спринт должен дать тестовую кампанию, заявку и карточку сделки с прозрачными статусами.